

Sensibilisation aux serveurs d'applications JavaEE

Jonathan Lejeune

Sorbonne Université/LIP6

SRCS – Master 1 SAR 2023/2024

sources :

Développons en Java, Jean-Michel Doudoux

Cours précédent de Lionel Seinturier

Wikipédia

Cours de Jérôme Hugues et Ada Diaconescu (Telecom ParisTech)

Un cycle de vie complexe

- Définition des interfaces, de la topologies de l'application
- Implantation des interfaces
- Tests
- Configuration
 - ⇒ Paramétrage des entités applicatives et de l'env. d'exécution
- Déploiement sur plusieurs nœuds
 - ⇒ Positionner les entités applicatives et les préparer à s'exécuter

Et des évolutions

- Correction de bugs
- Évolutions/extensions des interfaces
- Déploiement sur une nouvelle architecture (topologie, OS, ...)

Construire une application répartie

Les outils à notre disposition

- Remote Procedure Call
- Objets répartis
- Message Oriented Middleware

Leurs limites

Ces outils butent sur :

- le passage à l'échelle de l'application (nombre d'entités)
- l'évolution de l'application

Pas de standard de déploiement et de configuration

- Instanciation manuelle des objets sur le système
 - Politiques de configuration à plusieurs niveaux :
 - bus à objets, serveur d'activation, objet, connexion
- ⇒ Configuration et déploiement noyés dans le code applicatif

Solution : séparer le code d'une entité des mécanismes de configuration et de leur mise en place

Difficile d'étendre les fonctionnalités des objets

- Impossibilité de connaître les clients d'une interface
- ⇒ difficile d'assurer le suivi des modifications simplement

Solution : étendre le modèle de programmation pour ajouter la notion "d'interface requise" à celle "d'interface publiée"

Définition selon [Szyperski 97]

*Unité de composition qui a, par **contrat**, spécifié uniquement ses **interfaces** et ses **dépendances** explicites de contextes. Un composant logiciel peut être déployé indépendamment et est sujet à composition par des entités tierces.*

Une notion commune à plusieurs systèmes sous différentes formes

- Logicielle : EJB, Module java
- Modélisation : composants UML
- Matérielle : composants électronique

En résumé

Outil facilitant la **réutilisation**, la **composition** et le **déploiement**

Propriétés des composants

- unité de déploiement indépendante
 - ⇒ Séparation forte par rapport à l'environnement
 - ⇒ Déploiement partiel impossible
- unité de composition par une entité tierce
- unité qui n'a pas d'état persistant
 - ⇒ notion de copie non définie
 - ⇒ un composant est soit disponible soit indisponible

Propriétés des objets

- Unité d'instanciation ; il a un identifiant unique
 - ⇒ instanciation partielle impossible
 - ⇒ plan de construction : la classe
- Encapsule son état et son comportement
 - ⇒ État change, pas son identifiant
- L'état peut être persistant

Apports du composant sur l'objet

- plus haut niveau abstraction
- meilleure encapsulation, protection, autonomie
- communications plus explicites
port, interface, connecteur
- connectables/composables
- séparation de la partie métier de la partie technique
- meilleure couverture du cycle de vie
conception, implémentation, packaging, déploiement, exécution

Un nouveau modèle de programmation

Structuration d'une application répartie en définissant :

- Des composants implantant une fonctionnalité
- Un container définissant les frontières des composants
- Un framework à composants permettant l'automatisation de :
 - la connexion et l'assemblage
 - la configuration
 - l'instanciation et du déploiement



JAKARTA EE

(anciennement Java EE)

Qu'est ce que c'est ?

Un ensemble de concepts et de spécifications définies par Oracle pour le développement d'applications Java réparties pour les entreprises.

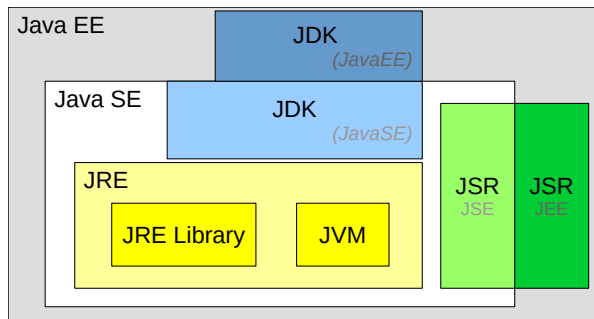
En pratique

- Une Extension de l'API de de Java Standard Edition (Java SE) :
 - correspondance objet Java et données relationnelles (SGBD)
 - services Web
 - architectures distribuées multi-tier
- Des composants modulaires s'exécutant dans un conteneur
- Un **serveur d'applications** automatisant interactions, dépendance et déploiement des conteneurs et des composants

Domaines applicatifs

E-commerce, Systèmes d'informations, Sites web, services,

Positionnement de Jakarta EE



Entités associées à chaque version de Java Enterprise Edition

- **Java Specification Requests (JSR)** : spécif. de la version considérée
- **Java Development Kit (JDK)** : les bibliothèques logicielles
- **Java Runtime Environment (JRE)** : le seul env. d'exécution

API de communications

- **Java RMI** : requête/réponse entre objets Java
- **JAX-RPC/JAX-WS/JAX-RS** : requête/réponse Web Service
- **JMS** : messages + boîte à lettres

API de connexion

- **JNDI** : annuaires (LDAP, DNS, ...), et espace de noms d'objet (ENC)
- **JDBC** : bases de données
- **JavaMail** : serveur mail

API de développement de composants d'application

- **EJB** : Composants distribués transactionnels
- **Servlet** et **Portlet** : Composants Web
- **JPA** : traduction objet et entités relationnelles (persistance)
- **JTA** : gestion de transactions

Implémentations existantes

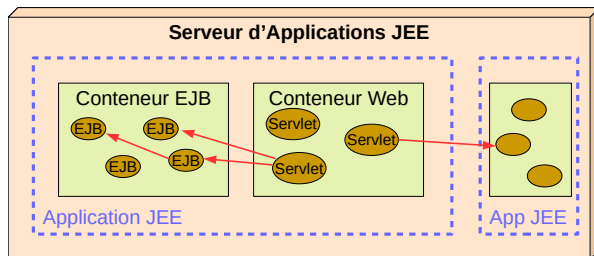
- Une implémentation de référence : Oracle GlassFish 6.2
- Commerciales : WebSphere (IBM), JEUS 8 (TmaxSoft), ...
- Open-source : Wildfly 12/JBoss (RedHat), Geronimo (Apache),

Processus de certification

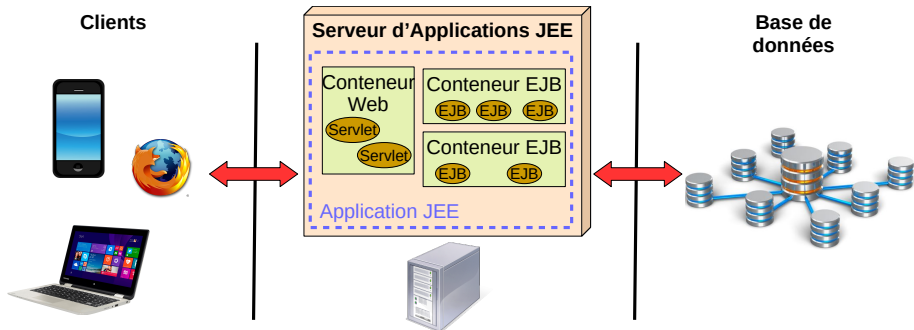
- Payant sauf pour plates-formes open-source
- Assez-lourd à mettre en œuvre (plusieurs dizaines de milliers de tests)
- Une certification par version de JavaEE

Caractéristiques

- Composée de conteneur(s) de composants (EJB, Servlet ...)
- Des connecteurs ou/et des dépendances entre composants
- L'ensemble archivé dans une **E**nterprise **A**pplication **a**rchive (EAR)
- Un serveur d'applications déployant et gérant le tout



EJB et architecture 3-tier



Adaptation parfaite pour les client légers

- **Présentation** : affichage et saisie des données par le client
- **Traitements** : le serveur d'appli. hébergeant les conteneurs d'EJB
- **Stockage** : une base de données dédiée pour la persistance des info

Caractéristiques

- Entité obligatoire pour exécuter un EJB
- Propose des services qui assure la gestion :
 - du cycle de vie du bean
 - de l'accès au bean via le réseau (notification auto à l'arrivée d'un message)
 - de la sécurité des accès (ex : cryptage)
 - des accès concurrents
 - de la synchronisation des données d'un objet avec une BD
 - du contrôle de la charge :
 - fabrique de nouvelles instances
 - surveillances de instances inactives
 - des transactions

Toute entité externe au serveur communiquent indirectement avec l'EJB
via le conteneur

API interdite d'utilisation par le programmeur

- Thread
- flux I/O
- code natif (méthode `native`)
- API graphique : AWT, Swing, JavaFx

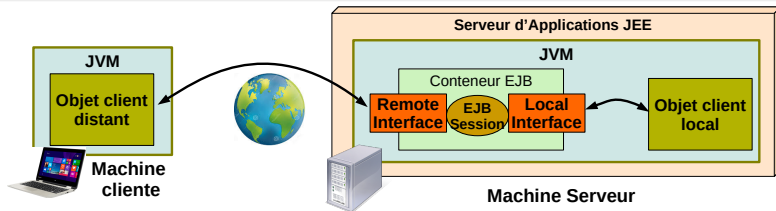
Seul le conteneur peut utiliser ces API

Les interfaces des EJB Session

Deux types d'interfaces

- Distant :
 - les services (méthodes) offerts par l'EJB à ses clients
 - Interface Java avec l'annotation `@Remote`
 - Utilisation du réseau + Java RMI
- Locale :
 - les services offerts par l'EJB aux clients présent dans la même JVM
 - Interface Java avec l'annotation `@Local`
 - Peut être identique ou différente à l'interface distante

⇒ Optimisation des performances car évite le réseau



Caractéristiques

- Sans état :
 - 1 instance par invocation
 - Pas d'information conservée entre 2 appels successifs
- Le conteneur gère un pool d'instance qui sont utilisées au besoin
⇒ s'adapte à la montée en charge

L'annotation @Stateless

S'utilise sur une classe Java et possède des attributs optionnels :

- `String name` : nom de l'EJB (défaut = nom de la classe)
- `String description` : description de l'EJB

Les annotations de méthodes de la classe (callback événementiel)

- `@PostConstruct` : appelé après la construction
- `@PreDestroy` : appelé avant la destruction

Les EJB Session stateless : un exemple

L'interface

@Remote

```
public interface CalculRemote{  
    public int add(int a, int b);  
}
```

@Local

```
public interface CalculLocal{  
    public int add(int a, int b);  
    public int minus(int a, int b); //service local uniquement  
}
```

La classe

@Sateless // ou bien par ex. @Stateless(name=calculette)

```
public class CalculBean implements CalculRemote, CalculLocal{  
    public int add(int a, int b){ return a + b; }  
    public int minus(int a, int b){ return a - b; }  
}
```


Caractéristiques

- Maintient d'états entre deux invocations d'un même client
 - 1 instance dédiée à chaque client
 - Expiration au bout d'un délai d'inactivité
- **Attention** : l'état n'est pas persistant
⇒ données perdues à la destruction du bean ou à l'arrêt du serveur

L'annotation @Stateful

S'utilise sur une classe Java et possède des attributs optionnels :

- String name : nom de l'EJB (défaut = nom de la classe)
- String description : description de l'EJB

L'annotation de méthode @Remove

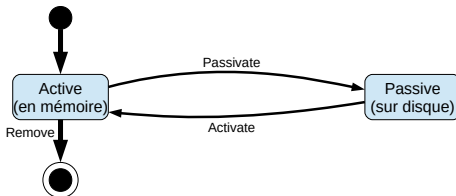
Indiquer un traitement à faire en fin de session.

Les EJB Session stateful : activation/désactivation

Principe

Le conteneur peut décider sérialiser/désérialiser sur disques des EJB Stateful pour gérer son espace mémoire

⇒ Similaire au mécanisme de swap d'un OS



Les annotations de méthodes de la classe (callback événementiel)

- `@PostConstruct`, `@PreDestroy`
- `@PostActivate`, `@PrePassivate`
- `@Remove`

Les EJB Session stateful : un exemple

```
@Sateful
public class CartBean implements CartRemote{
    private List<Object> items = new ArrayList<>();
    private List<Long> quantities = new ArrayList<>();

    public void addItem(int res, int qte){ .... }
    public void removeItem(int ref){....}

    @Remove
    public void confirmOrder(){ ... }
}
```

Caractéristiques

- Stateful ayant une seule instance pour tous les clients
- Apports :
 - Exécution de code au lancement ou à l'arrêt de l'application
 - Partage de données avec gestion des accès concurrents
- **Attention** : état conservé durant toute la durée de vie de l'application
⇒ données perdues à la fin de l'application ou de la JVM

Les annotations sur la classe

- @Singleton : déclaration de la classe comme singleton
- @Startup : si présent, la classe sera instanciée au lancement de l'application
Attention : n'impose pas d'ordre de lancement
- @DependsOn : permet de spécifier une dépendance avec d'autres EJB.
⇒ assure que le conteneur démarrera les dépendances avant

EJB Session singleton : exemple

```
@Singleton
@Startup
@DependsOn({"MonAutreBean"})
public class MonCache implements MonCacheRemote{
    private Map<String, Object> cache;

    @PostConstruct
    public void init(){
        this.cache=new HashMap<String, Object>();
    }

    @Override
    public Object get(String k){ return cache.get(k);}

    @Override
    public Object put(String k, Object v){ return cache.put(k,v);}

    @PreDestroy
    public void fin(){System.out.println("Requiem_Cachus");}
}
```

Stratégies Container Managed Concurrency (par def.)

Le conteneur gère les accès concurrents via l'annotation `@Lock`

- Chaque méthode possède un verrou de type read ou write
 - read : accessible par plusieurs thread simultanément
 - write (par def.) : méthode en exclusion mutuelle
- Si l'annotation est sur la classe alors ceci agit comme valeur par défaut sur toutes les méthodes de la classe.

Stratégies Bean Managed Concurrency

Accès concurrents à la charge du développeur (`synchronized`, `volatile`, ..)

La stratégie est précisée avec l'annotation de classe `@ConcurrencyManagement`

Nommer les EJB session avec JNDI

Attribution des noms

Tout EJB session est enregistré dans un annuaire avec un nom unique attribué automatiquement par le conteneur au moment du déploiement

Un format standard quelque soit le serveur d'application

`java:global/<app_name>/<module_name>/<bean_name>!<interface_name>`

- `app_name` : nom de l'application JavaEE dans laquelle le module de l'EJB est packagé
- `module_name` : nom du module dans lequel l'EJB est packagé
- `bean_name` : nom du bean (attribut *name* des annotations d'EJB session)
- `interface_name` : nom de l'interface exposée (interface remote)

Accès à un EJB session par un client (hors conteneur)

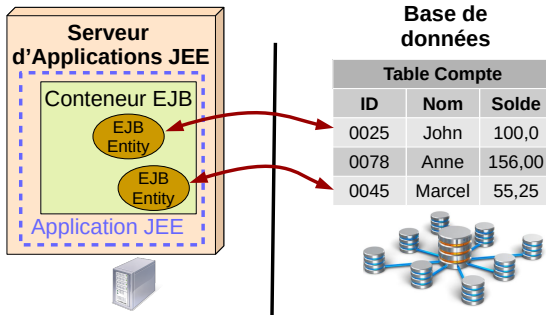
Principe

- Initialiser un contexte JNDI
- Récupérer une instance de mandataire d'une instance d'EJB Session avec lookup
- Caster l'instance dans le type de l'interface voulue (remote ou locale)

```
public class ClientEJB{  
    public static void main(String args[]){  
        Context context = new InitialContext();  
        String url="java:global/mon_appli/mon_module/Foo!FooRemote";  
        Object o = context.lookup(url);  
        FooRemote foo = (FooRemote) o;  
    }  
}
```


Caractéristiques

- Représentation d'une donnée manipulée par l'application
 - Stockée sur support persistant accessible via JDBC (ex : SGBD)
 - Permet une correspondance objet Java et Tuple relationnel
 - Possibilités de définir des clés, des relations, des recherches
⇒ on manipule des objets Java, plutôt que des requêtes SQL
- Mis en œuvre grâce aux annotations et à Java Persistence API (JPA)



EJB Entity : exemple

```
@Entity
public class Compte implements Serializable{

    private static final long serialVersionUID=1L;

    @Id
    @GeneratedValue
    private long id;
    private String nom;
    private double value;

    public Compte() {}

    public long getId(){return id;}
    public void setId(long id){this.id=id;}
    public long getNom(){return nom;}
    public void setNom(String nom){this.nom=nom;}
    public long getValue(){return value;}
    public void setValue(double value){this.value=value;}
}
```